Safety-Critical Learning of Robot Control with Temporal Logic Specifications

Mingyu Cai[1], Cristian-Ioan Vasile[2]

***Abstract*—Reinforcement learning (RL) is a promising control approach in many scenarios. However, safety-critical applications are still a challenge due to lack of safety guarantees during exploration and subsequent deployment. The learning problem becomes even more difficult for complex tasks with temporal and logical constraints. In this paper, we introduce a modular deep RL architecture as a control framework to satisfy complex tasks specified using Linear Temporal Logic (LTL). To enhance safety, we propose a safe "shield" to constrain RL actions within a safe set. In turn, a challenge with this strategy is that the safe filter can restrict RL exploration during training. To address this limitation, we have developed an innovative safe guiding process by integrating the properties of LTL automata with perturbations of the safe "shield". This innovation is verified to maintain the original optimality of LTL satisfaction and enhances exploration efficiency. Finally, we demonstrate that our approach consistently achieves near-perfect success rates and safety guarding with a high level of confidence during the training. The video demonstration can be found on our YouTube [1].**

***Keywords*—Formal Methods, Reinforcement Learning, Control Barrier Function, Gaussian Process**

I. INTRODUCTION

Reinforcement learning (RL) is a sequential decision-making process that maximizes the long-term reward via interaction with environments [1]. Markov decision processes (MDP) are often employed to model the dynamics of robots and their interaction with environments. Growing research has been devoted to studying RL-based motion planning without any prior knowledge of the complex robot dynamics and uncertainty models. This approach has been successfully employed in robotics, where it was extended to continuous state-action spaces via actor-critic methods [2], [3]. However, a key characteristic of RL is that it introduces noisy policies during the learning process while exploring the environment. The challenge of interpreting the inner workings of many RL algorithms makes it intractable to encode the behavior of the systems during training, especially while the learning parameters have yet to converge to a stable control policy. Thus, most modern RL algorithms have limited success on safety-critical systems.

The concept of control barrier function (CBF) was introduced as a pivotal instrument aimed at guaranteeing safety-critical constraints [4]. Recently, several methods have been proposed addressing the issue of model uncertainty in the safety-critical problems. The work [5] pre-specified the control barrier function and control Lyapunov function to achieve stability and safety, while learning the unknown part of dynamics. This strategy has also been extended using Gaussian Processes (GPs) for model identification [6]. These works achieve the objective of asymptotic stability relying on the control Lyapunov functions that are challenging to design manually. In contrast, we leverage deep RL for optimal control via interactions while ensuring safe exploration during learning.

In policy optimization algorithms [2], [3], RL agents are free to explore, but may engage in dangerous behaviors during learning, as long as it leads to performance improvement. Safe RL [7] is an emerging research field focusing on finding optimal policies that maximize the expected return while ensuring safe exploration, i.e., satisfying safety-critical constraints during the learning process [8]. As for unknown models, GPs [9] provide an efficient data-driven method for online non-parametric model estimation with probabilistic confidence. The work [10] utilizes GPs for reachability analysis and iterative predictions. By integrating GPs and CBF, existing works certify learning-based policies or dynamic systems while ensuring safe exploration [11]–[13]. However, all these works mainly focus on conventional, simple objectives rather than complex high-level specifications. In this framework, we tackle control problems under dynamic uncertainties, achieving temporal logic goals via a safe RL approach.

Contributions. This framework generalizes the deep RL-based approach to satisfy Linear Temporal Logic (LTL) specifications while maintaining safe exploration. By utilizing dense rewards crafted based on the automaton structure, we apply a modular deep RL architecture, allowing us to inspect and improve the performance of compositionally satisfying complex LTL specifications over infinite horizons. To monitor and correct the potentially unsafe RL control during training, a safe "shield" associated with an online model identification is developed through GPs and Exponential CBF (ECBF). The structure can be integrated with any actor-critic RL algorithm.

The core contributions can be summarized as: (i) We develop a safe deep RL framework for continuous state-action space to satisfy general LTL formulas over infinite horizons. (ii) To the best of our knowledge, this is the first effort to incorporate safety-critical specifications to improve RL exploration efficiency. (iii) The safe filter, combined with the exploration guidance module, is rigorously proven to maintain the original optimality of learned policies for LTL satisfaction.

Related Works. Recently, there has been increased interest in synthesizing optimal controllers for robot systems subject to several high-level temporal logics. Mature works [14], [15] abstract the interactions between robots and environments

[1]Department of Mechanical Engineering, University of California Riverside, CA, USA. Email: {mingyu.cai}@ucr.edu, [2]Department of Mechanical Engineering, Lehigh University, Bethlehem, PA, USA. Email: {cvasile}@lehigh.edu

[1]https://youtu.be/liB1Po7oXeo

subject to LTL for planning, decision-making, and optimization. However, all of them assume that dynamic systems are known to generate low-level navigation controllers. Assuming the robots' dynamics to be unknown, learning-based approaches are proposed in [16], [17] by taking automata of LTL as reward machines. Other works [18], [19] develop a reward scheme for infinite-horizon LTL satisfaction. To consider continuous space, previous work [20] proposes a provably correct framework leveraging deep neural networks. However, none of the above consider the safety-critical aspects during the learning process.

Considering safe RL-based control synthesis subject to LTL, Li et al. [21] first propose a safe RL using CBF guided by the robustness of Truncated LTL for satisfaction in dynamic environments. However, Truncated LTL can only express properties over finite horizons. Other related works [15], [22], [23] investigate LTL formulas that are abstraction-free. They all assume the robot dynamics are known and can follow the high-level planned paths by designing appropriate low-level controllers. Differently, our work tackles the nominal model that does not perfectly match the true dynamic system due to unknown disturbances.

II. PRELIMINARIES AND PROBLEM FORMULATION

The evolution of a dynamic system $\mathcal{S}$ starting from any initial state $s_0 \in S_0$ is given by

$$\dot{s} = f(s) + g(s)\, a + d(s), \tag{1}$$

where $s(t) \in S \subseteq \mathbb{R}^n$ is the state vector in the compact set S and $a(t) \in A \subseteq \mathbb{R}^m$ is the control input, at time t. The Lipschitz continuous functions $f : \mathbb{R}^n \to \mathbb{R}^n$ and $g : \mathbb{R}^n \to \mathbb{R}^{n\times m}$ define the known state dynamics, and d is a deterministic unknown disturbance function that is locally Lipschitz continuous. In (1), we assume the continuous function d can be uniformly approximated on a compact set S.

Let AP be a set of atomic propositions that define state properties of interest, and $L : S \to 2^{AP}$ the associated state labeling function. Given a state trajectory $s(t)$ obeying (1), we define the discrete-time output word $\mathbf{o} = o_0 o_1 \ldots$ such that $o_i = L(s(t_i))$, where $t_i = i \cdot \Delta t$ for a given discretization time step $\Delta t > 0$ for all $i \in \mathbb{Z}_{\geq 0}$.

Linear Temporal Logic (LTL) is a formal language to describe high-level system specifications. LTL formulas define Boolean and temporal properties over a set of atomic propositions AP. The syntax of an LTL formula is [24]

$$\phi ::= \text{true} \mid ap \mid \phi_1 \wedge \phi_2 \mid \neg\phi_1 \mid \bigcirc\phi \mid \phi_1 \mathcal{U} \phi_2 \,,$$

where $ap \in AP$ is an atomic proposition, $\neg$ and $\wedge$ are the Boolean negation and conjunction operators, and $\bigcirc$ and $\mathcal{U}$ are *next* and *until* temporal operators. Other propositional logic operators such as false, disjunction $\vee$, implication $\rightarrow$, and temporal operators always $\Box$, eventually $\Diamond$ are be derived from the operators above.

The semantics of an LTL formula are interpreted over words $\mathbf{o} = o_0 o_1 \ldots$, where $o_i \in 2^{AP}$ for all $i \geq 0$. We denote by $o \models \phi$ that the word o satisfies the LTL formula ϕ. Formal definitions and more details about LTL formulas can be found in [24]. In this article, we restrict our attention to LTL formulas that exclude the *next* temporal operator, which is not meaningful for continuous time execution since it may not be possible to require instantaneous motion [15], [25].

We capture interactions between a robot's motion governed by the dynamical system $\mathcal{S}$ and the environment as a continuous labeled Markov Decision Processes (cl-MDP).

Definition 1. A cl-MDP is a tuple $\mathcal{M} = (S, S_0, A, p_S, AP, L)$, where $S \subseteq \mathbb{R}^n$ is a continuous state space, S_0 is a set of initial states, $A \subseteq \mathbb{R}^m$ is a continuous action space, AP is a set of atomic propositions, $L : S \to 2^{AP}$ is a labeling function, and p_S represents the system dynamics. The distribution $p_S : \mathfrak{B}(\mathbb{R}^n) \times A \times S \to [0,1]$ is a Borel-measurable conditional transition kernel such that $p_S(\cdot \mid s, a)$ is a probability measure of the next state given current $s \in S$ and $a \in A$ over the Borel space $(\mathbb{R}^n, \mathfrak{B}(\mathbb{R}^n))$, where $\mathfrak{B}(\mathbb{R}^n)$ is the set of all Borel sets on $\mathbb{R}^n$. The transition probability p_S captures the motion uncertainties of the agent.

The abstraction cl-MDP $\mathcal{M}$ shares the same state and action spaces as the dynamical system $\mathcal{S}$. The continuous-time policy $a(\cdot)$ for system (1) is performed at triggering times and computed using policy search in $\mathcal{M}$ by the RL agent as a piecewise-constant function induced by the actions at times $t_i = i \cdot \Delta t$, $i \in \mathbb{Z}_{\geq 0}$ [3], [26].

For simplicity, we abuse notation and define s_i as the state at time t_i, i.e., $s_i = s(t_i)$, $i \geq 0$. We use similar notation for the actions, $a_i = a(t_i)$. A control policy for the cl-MDP generates piecewise-constant control input signals at each time-step for dynamics (1) i.e., $\boldsymbol{\xi} = \xi_0 \xi_1 \ldots$. It yields a path $\boldsymbol{s} = s_0 a_0 s_1 a_1 s_2 a_1 \ldots$ over $\mathcal{M}$ such that for each transition $s_i \xrightarrow{a_i} s_{i+1}$, a_i is generated based on ξ_i, where $s_i \xrightarrow{a_i} s_{i+1}$ has the property $p_S(s_{i+1} \mid s_i, a_i) > 0$.

Without information on p_S, RL [27] can be employed to optimize designed rewards and find the optimal policy. This framework focuses on addressing the continuous state-action space by employing the actor-critic method [3]. Moreover, our goal involves ensuring safety. Towards this, we introduce a LTL safety property observed at triggering times and add it to the specification to aid the RL search.

First, a safe set $\mathcal{C}$ is defined by the super-level set of a continuous differential (barrier) function $h : \mathbb{R}^n \to \mathbb{R}$,

$$\mathcal{C} = \{ s \in \mathbb{R}^n : h(s) \geq 0 \}. \tag{2}$$

The set $\mathcal{C}$ is forward invariant for system (1) if $\forall s(0) \in \mathcal{C}$, the condition $s(t) \in \mathcal{C}$, $\forall t \geq 0$, holds, which indicates the system (1) is safe. We assume RL-agents start in the safe set.

Let ϕ_{safe} denote the safety proposition of h associated with the safe set $\mathcal{C}$. Formally, $\phi_{safe} \in L(s_i)$ if $h(s_i) \geq 0$ or equivalently $s_i \in \mathcal{C}$. The safety-critical task is defined as $\Box\phi_{Safe}$, i.e., $\mathbf{o} \models \Box\phi_{Safe}$ if $s_i \in \mathcal{C}$ and $h(s_i) \geq 0$ for all $i \in \mathbb{Z}_{\geq 0}$, where $\mathbf{o} = o_0 o_1 \ldots$ and $o_i = L(s_i)$.

Consider an RL-agent that performs an LTL mission $\phi = \Box\phi_{safe} \wedge \phi_g$, where ϕ_g denotes a general high-level LTL task. Let $\boldsymbol{s}^{\boldsymbol{\xi}}_{\infty} = s_0 s_1 s_2 \ldots$ denote the induced path under a policy $\boldsymbol{\xi}$ and $L\left(\boldsymbol{s}^{\boldsymbol{\xi}}_{\infty}\right) = l_0 l_1 \ldots$ be the sequence of labels associated with $\boldsymbol{s}^{\boldsymbol{\xi}}_{\infty}$ such that $l_i = L(s_i), \forall i \in \{1, 2, \ldots\}$. We can evaluate the LTL satisfaction from $L(\boldsymbol{s}^{\boldsymbol{\xi}}_{\infty})$. Then, the probability of LTL satisfactoin under a policy can be measured according to [24].

Assumption 1. *It is assumed that there exists at least one policy satisfying the task ϕ with non-zero probability.*

Assumption 1 also indicates that there are no conflicts between $\Box\phi_{safe}$ of h and ϕ_g. Motivated by robot missions such as planetary exploration (see Sec. V) that require them to safely learn to perform complex, long-horizon tasks under unknown disturbances, we formulate the LTL control synthesis as the learning problem

Problem 1. Given a cl-MDP $\mathcal{M}$ with unknown transition probabilities p_S, and an LTL task $\phi = \Box\phi_{safe} \wedge \phi_g$ with corresponding safe set $\mathcal{C}$,
(i) learn an optimal policy $\boldsymbol{\xi}^*$ that maximizes the probability of LTL satisfaction in the limit,
(ii) and maintain the satisfaction of safety-critical task $\Box\phi_{safe}$ during the learning process (exploration) and policy execution.

We solve the problem by developing automata based rewards of deep RL for LTL satisfaction for part (i) in Section III, coupled with a safety filter and exploration guidance for part (ii) in Section IV.

III. RL BASED LTL SATISFACTION

A. Automaton-based LTL Satisfaction

An LTL formula ϕ can be converted into an automaton to verify satisfaction [24]. When applying RL for LTL satisfaction, previous works [20], [28] have shown that Embedded Limit Deterministic (Generalized) Büchi Automaton (E-LDGBA) extending [29] yield denser automaton-based rewards and provide more effective learning-based LTL satisfaction compared with other automata. To simplify the notations, we omit the embedding structure of E-LDGBA and only introduce the main ingredients related to RL rewards design and exploration guidance. More details can be found in [28].

Definition 2. An E-LDGBA is a tuple $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$, where Q is a finite set of states, $\Sigma = 2^{AP}$ is a finite alphabet, $q_0 \in Q$ is an initial state, and $F = \{F_1, F_2, \ldots, F_f\}$ is a set of accepting sets with $F_i \subseteq Q$, $\forall i \in \{1, \ldots, f\}$, $\delta : Q \times (\Sigma \cup \{\epsilon\}) \to 2^Q$ is the transition function.

Denote by $\boldsymbol{q} = q_0 q_1 \ldots$ a run of a E-LDGBA, where $q_i \in Q$, $i = 0, 1, \ldots$. The run $\boldsymbol{q}$ is accepted by the E-LDGBA, if it satisfies the generalized Büchi acceptance condition, i.e., $\inf(\boldsymbol{q}) \cap F_i \neq \emptyset$, $\forall i \in \{1, \ldots f\}$, where $\inf(\boldsymbol{q})$ denotes the set of states that repeat infinitely often in $\boldsymbol{q}$.

We denote Q_{sinks} as the union of all non-accepting sink components of an E-LDGBA. Each non-accepting sink component is a strongly connected directed graph with no outgoing transitions s.t. the acceptance condition can not be satisfied. Thus, a trace reaching Q_{sinks} is doomed to not satisfy the given LTL property. The set of *non-accepting unsafe states* $Q_{unsafe} \subseteq Q_{sinks}$ of an E-LDGBA is a set of sink states s.t. $Q_{unsafe} = \left\{(q' \in Q) \middle| \forall q \in Q, q' = \delta(q, \neg\phi_{safe})\right\}$. The automaton system enters Q_{unsafe} whenever $\Box\phi_{safe}$ is violated. The set Q_{unsafe} can be used as an indicator of unsafety during learning process.

To satisfy a complex LTL-defined task over infinite horizons, we can introduce a product structure.

Definition 3. Given a cl-MDP $\mathcal{M}$ and an E-LDGBA $\mathcal{A}_\phi$, the Product MDP (P-MDP) is defined as $\mathcal{P} = \mathcal{M} \times \mathcal{A}_\phi = (X, U^{\mathcal{P}}, p^{\mathcal{P}}, x_0, F^{\mathcal{P}}, f_V)$, where $X = S \times Q$ is the set of product states, i.e., $x = (s, q) \in X$; $U^{\mathcal{P}} = A \cup \{\epsilon\}$ is the set of actions, where the ϵ-actions are only allowed for transitions from Q_N to Q_D; $x_0 = (s_0, q_0)$ is the initial state; $F^{\mathcal{P}} = \left\{F_1^{\mathcal{P}}, F_2^{\mathcal{P}} \ldots F_f^{\mathcal{P}}\right\}$ where $F_j^{\mathcal{P}} = \{(s, q) \in X \mid q \in F_j, j \in \{1, \ldots, f\}\}$; $p^{\mathcal{P}}$ is the transition kernel for any transition $\left(x, u^{\mathcal{P}}, x'\right)$ with $x = (s, q)$ and $x' = (s', q')$ such that : (1) $p^{\mathcal{P}}(x, u^{\mathcal{P}}, x') = p_S(s' | s, a)$ if $s' \backsim p_S(\cdot | s, a)$, $\delta(q, L(s)) = q'$ where $u^{\mathcal{P}} = a \in A$ (2) $p^{\mathcal{P}}(x, u^{\mathcal{P}}, x') = 1$ if $u^{\mathcal{P}} \in \{\epsilon\}$, $q' \in \delta(q, \epsilon)$ and $s' = s$; and (3) $p^{\mathcal{P}}(x, u^{\mathcal{P}}, x') = 0$ otherwise.

The P-MDP $\mathcal{P}$ captures the identification of admissible agent motions over $\mathcal{M}$ that satisfy the task ϕ. Let $\boldsymbol{\pi}$ denote a policy over $\mathcal{P}$. Any memory-less policy $\boldsymbol{\pi}$ of $\mathcal{P}$ can be projected onto $\mathcal{M}$ to obtain a finite-memory policy $\boldsymbol{\xi}$ [24]. In Problem 1, finding a policy $\boldsymbol{\xi}$ of $\mathcal{M}$ to satisfy ϕ is equivalent to searching for a policy $\boldsymbol{\pi}$ of $\mathcal{P}$ to satisfy the acceptance condition.

Remark 1. Explicitly constructing the P-MDP is impossible over continuous space. In this work, we generate P-MDP on-the-fly which means the approach is scalable to track the states of an underlying structure based on Def. 3.

To monitor the safety-critical requirement via automaton structure, we define the sink components of violating $\Box\phi_{safe}$ in an P-MDP as $X_{unsafe} \subseteq X$ s.t. $X_{unsafe} = \{(s, q) \in X) \mid q \in Q_{unsafe}\}$. If the system enters X_{unsafe}, it implies the violation of the safety constraint over $\mathcal{P}$.

B. Modular Deep Reinforcement Learning

To find the policies satisfying the acceptance condition of P-MDP with maximum probabilities, we apply the deep RL reward design in prior work [20] as the base reward scheme. For each transition $\left(x, u^{\mathcal{P}}, x'\right)$ in the P-MDP, we denote the reward and discounting functions as $R\left(x, u^{\mathcal{P}}, x'\right) = R\left(x\right)$ and $\gamma\left(x, u^{\mathcal{P}}, x'\right) = \gamma\left(x\right)$. Such a design only assigns a positive reward value to the accepting state $x \in F_i^{\mathcal{P}}$, the reward signal becomes sparse for the non-accepting state $x \notin F_i^{\mathcal{P}}$. To further increase the density of the reward, we apply a potential function $\Phi : X \to \mathbb{R}$ based on [30] that has no impact on the optimality of the base reward scheme, and transform the reward as:

$$R'\left(x, u^{\mathcal{P}}, x'\right) = R\left(x\right) + \gamma\left(x\right) \cdot \Phi\left(x'\right) - \Phi\left(x\right) \tag{3}$$

We design the potential function $\Phi(x)$ and $\gamma(x)$ based on the prior work [20] to improve the efficiency of exploration. The intuition of such a design is to assign positive values for unvisited automaton states. Thus, any RL policy maximizing expected return using the shaped reward (3) also maximizes the probability of satisfying ϕ. We can directly use a deep RL algorithm to find optimal policies over continuous spaces.

Based on the reward shaping, our framework constructs a modular deep RL architecture to reduce the global variance of the policy gradient algorithm and improve the performance of satisfying complex tasks. The intuitive idea is to divide the LTL task into several sub-tasks based on its automaton states of E-LDGBA and apply distributed deep RL for each sub-task.

That is, each state of the automaton in the E-LDGBA is a module and each transition between these automaton states is a "task divider". In particular, given ϕ and its E-LDGBA $\mathcal{A}_\phi$, we propose a modular architecture of $|Q|$ actor-critic structures, i.e., $\boldsymbol{\pi}_{q_i}\left(x\,|\theta^u\right)$ and $Q_{q_i}\left(x, u^{\mathcal{P}}\,|\theta^Q\right)$ with $q_i \in Q$, along with their replay buffer B_{q_i} acting as a data memory to save the RL-agent's transitions $\left(x, u^{\mathcal{P}}, x', R', \gamma\right)$. The individual ith actor-critic structure $Q_{q_i}\left(x, u^{\mathcal{P}}\,|\theta^Q\right)$ of each LTL stage can be selected based on the automaton component of current P-MDP state $x = (s, q_i)$ during training and testing. An example can be found in Fig. 3 of supplementary materials.

Remark 2. We can concatenate finite policies to satisfy the infinite-horizon task by decomposing a complex global LTL task into subtasks. Another benefit of the modular structure is to inspect and improve success rates compositionally.

IV. Exploration Guidued Safe RL

Up to this point, we have demonstrated the application of RL for LTL satisfaction without accounting for safety during learning. This section introduces the development of a safety guard as a "shield" by integrating Gaussian Processes (GPs) to approximate the unknown model in (1) and employing an Exponential Control Barrier Function (ECBF). Additionally, we propose an exploration guidance strategy to enhance optimal performance and efficiency.

A. Model-based Safe Filter

Gaussian Processes (GPs) are non-parametric regression methods to approximate the unknown system dynamics and their uncertainties from data [31]. We can use GP regression to identify the unknown disturbance function of the safe constraint derived from the unknown term $d(s)$ in (1). The main advantage of applying GPs compared with the feedforward neural network of nonlinear regressions is the quantifiable confidence of predictions. Informally, a GP is a distribution over functions, and each component $d_i(s)$ of n-dimensional $d(s)$ can be approximated by a GP distribution denoted as $\mathcal{GP}$ with a mean function $u_i(s)$ and a covariance kernel function $k_i(s, s')$ which measures similarity between any two states, i.e., $d_i(s) \sim \mathcal{GP}(u_i(s), k_i(s, s'))$. The class of the prior mean function and covariance kernel function is selected to characterize the model.

Any unknown function $d(s)$ is estimated over a multivariate GP with mean $u(s) = [\hat{u}_i(s), \ldots, \hat{u}_n(s)]^T$ and standard deviation $\sigma(s) = [\hat{\sigma}_i(s), \ldots, \hat{\sigma}_n(s)]^T$. The model estimation error is bounded with probability $(1-\delta)^n$ as:

$$\Pr\left\{|d(s) - u(s)| \leq k_\delta \cdot \sigma(s)\right\} \geq (1-\delta)^n, \tag{4}$$

where $\Pr\{\cdot\}$ represents the measurement probability, and k_δ is a design a parameter determining δ [31].

We can alleviate the expensive matrix computation of GPs via the episodic sampling method [11]. Any other methods for model estimation can be also used with our framework.

For continuous-time systems, control barrier function is an efficient tool for maintaining safety [4] that assumes to be relative-degree one. To enforce arbitrarily CBF constraints with high relative-degree, exponential control barrier function (ECBF) is introduced [32]. For a continuous differentiable function h with high relative degree $r_b \geq 1$, denote $f'(s) = f(s) + d(s)$ and the r_b^{th} time-derivative of $h(s)$ is $h^{(r_b)}$.

Definition 4. [32] Assume $d(x)$ is known in the system $\mathcal{S}$ (1), and the safe set $\mathcal{C} \subseteq S$ with a continuous differentiable (barrier) function $h : S \to \mathbb{R}$ in (2) with relative degree $r_b \geq 1$. $h(s)$ is an ECBF if there exists $K_b \in \mathbb{R}^{1\times r_b}$ s.t.

$$\sup_a [L_{f'}^{r_b} h(s) + L_g L_{f'}^{r_b-1} h(s) a + K_b \xi] \geq 0 \tag{5}$$

where $\xi = [h(s), L_{f'}h(s), \ldots, L_{f'}^{r_b-1}h(s)]^T$ is the Lie derivative vector, and the row vector K_b should be selected to render a stable close-loop matrix for the linearized system of ξ. The system $\mathcal{S}$ is forward invariant in $\mathcal{C}$ if starting from $s_0 \in \mathcal{C}$, there exists an ECBF $h(s)$ and controllers satisfy (5).

Now, we extend results to the system $\mathcal{S}$ with unknown term $d(x)$ by incorporating GPs. Let $B^{(r)}(s,a) = L_{f'}^{r_b} h(s) + L_g L_{f'}^{r_b-1} h(s) a + K_b \xi_b$ denote the true output of ECBF where $\hat{\xi}_b = [h(s), L_f h(s), \ldots, L_f^{r_b-1} h(s)]^T$ is the traverse variable, and $\hat{B^{(r)}}(s,a) = L_f^{r_b} h(s) + L_g L_f^{r_b-1} h(s) a + K_b \xi_b$ denote the ECBF for known dynamics $\dot{s} = f(s) + g(s)a$. Same as [5], we have the relation $B^{(r)}(s,a) = \hat{B^{(r)}}(s,a) + \Delta(s,a)$, where $\Delta(s,a)$ is the unknown mismatch term and can be approximated by the learned GP model with mean $u(s,a)$, covariance $\sigma(s,a)$ and k_δ. Here $B^{(r)}(s,a)$ is computed using numerical differentiation. We also have $\Delta(s,a) \in [u(s,a) - k_\delta\sigma(s,a), u(s,a) + k_\delta\sigma(s,a)]$, which allows to obtain the lower bound of ECBF as $B^{(r)}(s,a) \geq \hat{B^{(r)}}(s,a) + u(s,a) - k_\delta\sigma(s,a)$.

Theorem 1. *Consider the system $\mathcal{S}$ in* (1) *with unknown $d(x)$ and $\mathcal{C} \subseteq S$ with a differentiable (barrier) function $h : S \to \mathbb{R}$ s.t. $\forall s \in \mathcal{C}, h(s) \geq 0$. If a close-loop stable K_b exists s.t.*

$$L_f^{r_b} h(s) + L_g L_f^{r_b-1} h(s) a + K_b \xi_b + u(s,a) - k_\delta\sigma(s,a) \geq 0, \tag{6}$$

then starting from any $s_0 \in \mathcal{C}$, controllers of the system $\mathcal{S}$ satisfying (6)*, render the set $\mathcal{C}$ forward invariant with probability at least $(1-\delta)^n$.*

Proof. The proof directly follows the property of GPs as:

$$\Pr\left\{\mathcal{GP}_l \leq F(s,a) \leq \mathcal{GP}_h\right\} \geq (1-\delta)^n$$

where $F(s,a) = \hat{B^{(r)}}(s,a) + \Delta(s,a)$, $\mathcal{GP}_l = \hat{B^{(r)}}(s,a) + u(s) - k_\delta\sigma(s)$, and $\mathcal{GP}_h = \hat{B^{(r)}}(s,a) + u(s) + k_\delta\sigma(s)$. □

Next, we formulate the relaxed ECBF condition (6) as the quadratically constrained quadratic program (QCQP)

$$\begin{aligned} (a_{CBF}, \epsilon) &= \arg\min_{a,\epsilon} (\tfrac{1}{2} a^T H(s) a + K_\epsilon \epsilon) \\ \text{s.t} \quad & \hat{B^{(r)}}(s,a) + u(s,a) - k_\delta\sigma(s,a) + \epsilon \geq 0, \end{aligned} \tag{7}$$

where $H(s)$ is a positive definite matrix (pointwise in s), control limitations can be also added as another constraint. To ensure the existence of solutions for the QCQP, ϵ is a relaxation variable, and K_ϵ is a large value that penalizes safety violations. The solution of the relaxed ECBF-QCQP enforces the safe condition with minimum norm.

Remark 3. The development of a valid Exponential Control Barrier Function (ECBF) with a formal safety guarantee remains a largely uncharted territory. This challenge becomes even more formidable when there is a requirement to do so without compromising on conservativeness. In our specific scenario, this level of conservativeness can impede the fulfillment of Linear Temporal Logic (LTL) requirements.

One can easily extend the framework for data-driven method to learn barrier functions [33]. This work focuses on efficient safe learning for LTL satisfaction and leaves the exploration of learning valid ECBFs for future research. Also, we can apply multiple ECBFs as constraints into (7).

ECBF-GP based Safe Filter. Let $x_t = (s_t, q_t)$ and $\boldsymbol{\pi}_t$ denote the product state and learned policy at time-step t, respectively. The action $u_t^{\mathcal{P}}$ is obtained based on the policy $\boldsymbol{\pi}_t$, i.e., $u_t^{\mathcal{P}} \sim \boldsymbol{\pi}_t$, which can be projected to obtain $a_{RL}(s_t)$ based on Def. 3. However, such a controller may not be safe. To overcome this issue, we build the QCQP according to GP-ECBF in (7) as a safeguard module to provide the minimal perturbation $a_{pt}(s)$ for the original control $a_{RL}(s)$.

$$\begin{aligned}(a_{pt}, \epsilon) = \arg\min_{a,\epsilon} \ & \frac{1}{2}(a_{RL}(s)+a)^T H(s)(a_{RL}(s)+a) + K_\epsilon \epsilon \\ \text{s.t} \quad & \hat{B^{(r)}}(s,a) + u(s,a) - k_\delta \sigma(s,a) + \epsilon \geq 0,\end{aligned} \tag{8}$$

Consequently, the actual implemented safety-critical controllers use a_{pt} from (8) as

$$a_{Safe}(s) = a_{RL}(s) + a_{pt}(s).$$

Thus, the "shield" of (8) compensates the model-free RL controller $a_{RL}(s)$, and keeps the state safe via deploying the final safe controller $a_{Safe}(s)$.

B. Exploration Guidance

However, purely applying a safety shield in Section IV during the learning process may negatively influence the exploration and the original optimal convergence shown in Section III-B. Since the optimal parameterized RL-policy $\boldsymbol{\pi}^*$ of the controller $a_{RL}(s)$ is synthesized by optimizing the expected return, the feedback reward at each time should correspond to the controller $a_{RL}(s)$. While the actual reward collection in the reply buffer is associated with the controller $a_{CBF}(s)$, which is not consistent with the RL-policy $\boldsymbol{\pi}^*$ and induces undesired behaviors. As a result, the modular DDPG receives no informative feedback about the unsafe behaviors compensated via (8). Moreover, another crucial issue is that the functionality of $a_{Safe}(s)$ is too monotonous to guide policy exploration. The corresponding RL policy $\boldsymbol{\pi}^*$ may always linger around the margins of unsafe sets, as illustrated in Fig. 4 (a) of supplementary materials.

To improve exploration efficiency and preserve the original formal optimality of LTL satisfaction, this section proposes an automaton-based guiding strategy that enhances RL policies being explored within the set of safe guidelines during training. In particular, we utilize the properties of E-LDGBA and ECBF. FOr an LTL formula of the form $\phi = \Box\phi_{safe} \wedge \phi_g$, where $\Box\phi_{safe}$ with the safe set $\mathcal{C}$, the intuition for (8) is that $a_{pt}(s) \neq 0$ implies the RL controller $a_{RL}(s)$ is unsafe, i.e., it violates $\Box\phi_{safe}$ in ϕ. Consequently, the objective of efficient exploration is to encourage the RL-agent operating in the safe region $\mathcal{C}$, and to enforce $a_{pt}(s)$ decaying to zero.

Recall during the learning process, we have $x_t = (s_t, q_t)$, $\boldsymbol{\pi}_t$ at time t, and obtain $a_{RL}(s_t)$ as the corresponding $u_t^{\mathcal{P}} \sim \boldsymbol{\pi}_t$. GP-based QCQP (8) generates $a_{pt}(s_t)$ and $a_{Safe}(s_t)$. The next state $x_{t+1} = (s_{t+1}, q_{t+1})$ is generated by taking the safe action $a_{Safe}(s_t)$ s.t. $p_S(s_{t+1}|\, s_t, a_{Safe}(s_t)) \neq 0$ and $q_{t+1} = \delta(q_t, L(s_t))$. The automaton-based safe guiding consists of three steps: ECBF-based reward shaping, violation-based automaton updating, and relay buffer switching. The procedure of safe guiding is shown in Fig. 1 of supplementary materials.

Exploration Guidance. Given the transition $\delta_t^{\mathcal{P}} = (x_t, a_{Safe}(s_t), x_{t+1})$, where $x_t = (s_t, q_t)$, $a_{RL}(s_t)$ is obtained from $u_t^{\mathcal{P}} \sim \boldsymbol{\pi}_t$, $a_{Safe}(s_t) = a_{RL}(s_t) + a_{pt}(s_t)$, and $x_{t+1} = (s_{t+1}, q_{t+1})$ is generated from Def. 3 during learning process, the three-step exploration guiding is defined as
(1) The reward is shaped based on the safe properties:

$$R_{CBF}\left(\delta_t^{\mathcal{P}}\right) = \begin{cases} r_n \cdot |a_{pt}(s_t)|, & \text{if } a_{pt}(s_t) \neq 0, \\ R'\left(\delta_t^{\mathcal{P}}\right), & \text{if } a_{pt}(s_t) = 0, \end{cases} \tag{9}$$

where $r_n \in \mathbb{R}$ is a negative constant parameter s.t. $r_n < 0$;
(2) The automaton component q_{t+1} of product state x_{t+1} is updated:

$$q_{t+1} = \begin{cases} q \text{ s.t. } q \in Q_{unsafe}, & \text{if } a_{pt}(s_t) \neq 0 \\ \delta(q_t, L(s_t)), & \text{if } a_{pt}(s_t) = 0. \end{cases} \tag{10}$$

(3) Instead of storing information $a_{Safe}(s_t)$ and $R'\left(\delta_t^{\mathcal{P}}\right)$, add $\left(x_t, a_{RL}(s_t), R_{CBF}\left(\delta_t^{\mathcal{P}}\right), x_{t+1}\right)$ to replay buffer for training.

In (9), $r_n < 0$ s.t. $r_n \cdot |a_{pt}(s_t)|$ represents how much the $a_{RL}(s_t)$ violates the safety constraint $\Box\phi_{safe}$, i.e., propositional to the absolute value of CBF compensators. Similarly, (10) switches the automaton state $\overline{q}_{t+1}$ to an unsafe state in $\overline{Q}_{unsafe}$. Different from [11] that takes $a_{Safe}(s_t)$ into the replay buffer for training, the third step keeps the original modular RL controller $a_{RL}(s_t)$ and integrates it with the shaped reward R_{CBF} into training. As for actor-critic methods of the modular RL, such a design can improve the efficiency of exploration and stabilize the learning results since the controllers a_{RL} in the relay buffer are generated from the policy distributions of the actor, whereas controllers a_{Safe} are only for safe execution and are not consistent with the outputs of the modular actor-critic architecture. Finally, we show that the original optimal policies generated in Section III-B remain invariant via the safe learning and guiding processes.

Theorem 2. *Given a cl-MDP $\mathcal{M}$ and an E-LDGBA $\mathcal{A}_\phi$, the optimal policy $\boldsymbol{\pi}^*$ generated by any deep RL algorithm with three-step Exploration Guidance maximizes the probability of satisfying ϕ in the limit.*

Proof. Exploration Guidance efficiently enhances the rapid decaying of $a_{pt}(s_t)$ to 0. Its intuition is to bridge the connection between the unsafe component $\overline{Q}_{unsafe}$ of E-LDGBA and ECBF controllers $a_{pt}(s_t)$ s.t. $a_{pt}(s_t 0 \neq 0$ indicates the original $a_{RL}(s_t)$ violates the safe requirement of $\phi = \Box\phi_{safe} \wedge \phi_g$.

Table I: Baseline variants tested in case study.

Baseline	Modular	Safe Module	Exploration Guiding
Modular-(DDPG)-ECBF (On)-Guiding	✓	✓	✓
Modular-(DDPG)-ECBF Off-Guiding	✓	✓	X
Modular-DDPG	✓	X	X
Standard-DDPG	X	X	X

Consequently, the objective is to verify that assigning negative reward to the states x with automaton components $q \in \overline{Q}_{unsafe}$ preserves original optimal solutions using the reward design (3). We prove this by contradiction in Arxiv [34]. □

Exploration Guidance integrates the property of violation of the LTL formula $\phi = \Box\phi_{safe} \wedge \phi_g$ and the safe set to ensure efficient exploration. Thus, $a_{pt}(s_t)$ gradually decays to 0 and becomes inactive. The overall structure pushes the RL policies generated from the set of safe polices without altering the original optimality, while maintaining safety during learning. In the supplementary materials, the concept of enforcing optimal policies is visually depicted in Fig. 5, and the the overall procedure of safe learning and guiding is outlined in Alg. 1.

Lemma 1. *Given a cl-MDP $\mathcal{M}$ and an E-LDGBA $\overline{\mathcal{A}}_\phi$, the optimal policy $\boldsymbol{\pi}^*$ by the modular structure of deep RL maximizes the probability of satisfying ϕ in the limit.*

Proof. Lemma 1 indicates that the conclusion of Theorem 2 still holds when replacing standard deep RL algorithms into their modular structure based on automaton states as shown in Section III-B. It can be verified by formulating the subtask of each actor-critic structure as a safe goal-reaching task $\phi = \Box\phi_{safe} \wedge \Diamond\phi_{goal}$, since a valid transition of an E-LDGBA requires the RL-agent to eventually visit an area of interest. Per Theorem 2, such a subtask is satisfied with the maximum probability in the limit, which concludes Lemma 1. The detailed proof can be found in Arxiv [34]. □

V. EXPERIMENTS

We demonstrate the framework in several environments with corresponding LTL tasks and use Deep Deterministic Policy Gradient (DDPG) [3] as the backbone deep RL algorithm. We compare our framework referred to as safe modular with guiding (Modular-DDPG-ECBF-Guiding) with other three baselines summarized in Table I. It is worth pointing out that it is more challenging for RL agents to satisfy tasks over infinite horizons. Consequently, safety rates and success rates of all tasks are shown in Fig. 7 and 8, respectively. Rich experimental results can be found in supplementary materials.

We first tested our algorithms on controlling two systems in simulated OpenAI gym environments. The LTL formulas over infinite horizons are of the form $\phi_{g_1} = \Box\Diamond \mathtt{R}_{\mathtt{green}} \wedge \Box\Diamond \mathtt{R}_{\mathtt{yellow}}$, which require visiting the green and yellow regions infinitely often while staying within the safe set ϕ_{safe}. The tasks over finite horizons have the form $\phi_{g_2} = \Diamond\left(\mathtt{R}_{\mathtt{green}} \wedge \Diamond \mathtt{R}_{\mathtt{yellow}}\right)$.

Cart-Pole: The physical simulation of Cart-Pole is shown in Fig. 1. A pendulum is attached to a cart that moves horizontally along a frictionless track. The control input is a horizontal force on the cart. Denote $s = \left[\theta_p, s_c, \dot{\theta}_p, \dot{s}_c\right]^T$ as the state (cart position, pendulum angle and corresponding velocities) of the dynamic system. Its true dynamics are defined as follows:

$$\ddot{\theta}_p = \frac{(u-d) + m_1 \cos\theta_p \sin\theta_p + m_2 \dot{\theta}_p^2 \sin\theta_p}{m_3 + m_4 \cos^2\theta},$$

$$\ddot{s}_c = \frac{\kappa_1(u-d)\cos\theta_p + \kappa_2 \dot{\theta}_p^2 \cos\theta_p \sin\theta_p + \kappa_3 \sin\theta}{\kappa_4 + \kappa_5 \cos^2\theta_p},$$

where external control force u is limited to $u \in [-20N, 20N]$, and $m_1, \ldots, m_4$ and $\kappa_1, \ldots, \kappa_4$ are the physical parameters. To introduce model uncertainty, we add 30% error in the physical constants. The safe set consists of two barrier functions

$$\mathcal{C}_1 = \left\{(\theta_p, s_c) : (12^2 - \theta_p^2 \geq 0) \wedge (2.4^2 - s_c^2 \geq 0)\right\}.$$

The results of $\phi_{\mathtt{Cart1}}$ are shown in Fig. 2 and 3. Fig.2 compares the mean reward achieved via different baselines. It shows modular DDPG has better performance than the standard DDPG (higher rewards), and exploration guidance can improve the training efficiency. Fig.3 shows the absolute value of the maximum angle and position in each episode. It compares the

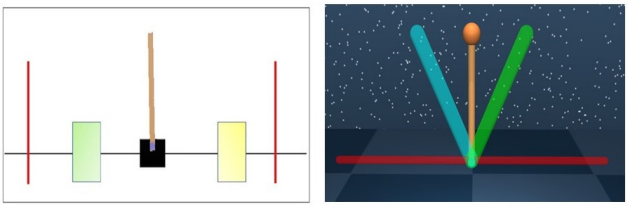

Figure 1: LTL specifications require the Cart-Pole to visit the green and yellow rectangular regions for (Left) Cart-Pole and (Right) Interted-Pendulum.

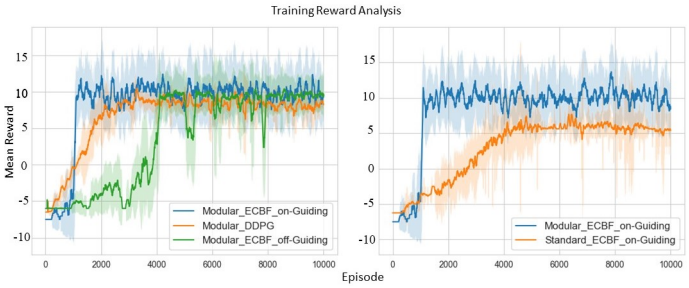

Figure 2: Cart-Pole: Mean rewards of the LTL formula $\phi_{\mathtt{Cart1}}$.

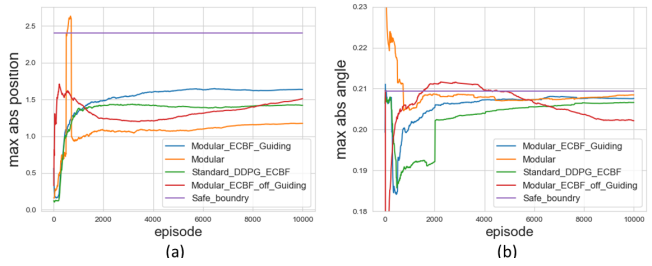

Figure 3: Cart-Pole: Maximum absolute value of positions (a) and angle (b) of the LTL formula $\phi_{\mathtt{Cart1}}$ for different baselines during evaluation process.

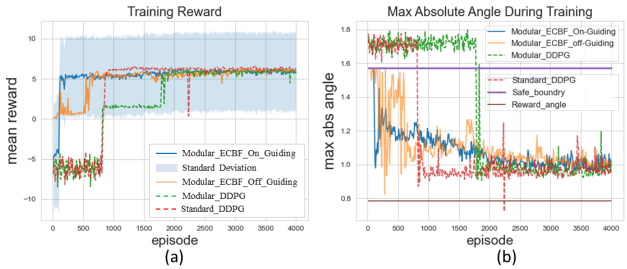

Figure 4: Inverted-Pendulum: reward collection and maximum absolute angle for the formula $\phi_{\texttt{Pen1}}$. (a) Mean reward. (b) Maximum absolute angle.

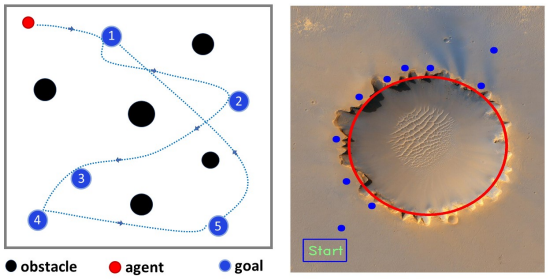

Figure 5: (left) The trajectory of particle gym for the task φ_{Gym_1}. (right) Mars exploration environment.

safe performance against the baselines. Especially, it shows the advantage of the exploration guidance. During training, since the dynamics of Cart-Pole is more complex and sensitive to the safety-critical task, the RL controllers always steer the systems close to the margin of the safe set.

Inverted-Pendulum: The physical simulation of inverted-pendulum is shown in Fig. 1. The true system dynamics of state θ with mass m, length l and torque u is $\ddot{\theta} = \frac{3g}{2l}\sin\theta + \frac{3}{2ml^2}u$, where torque is limited to $u \in [-15, 15]$ and the nominal model has 40% error in the physical parameters. The safe set is composed of a control barrier function $\mathcal{C}_2 = \left\{\theta : \frac{\pi}{2}^2 - \theta^2 \geq 0\right\}$. The LTL formula that holds the system in the safe set $\mathcal{C}_2$ is $\phi_{safe} = \phi_{\mathcal{C}_2}$. The full LTL formula over infinite horizon is $\phi_{\texttt{Pen1}} = \Box\phi_{\mathcal{C}_2} \wedge \phi_{g_1}$, where the blue and green regions are located at $-\frac{\pi}{4}$ rads and $\frac{\pi}{4}$ rads, respectively. The LTL formula over finite horizon is $\phi_{\texttt{Pen2}} = \Box\phi_{\mathcal{C}_2} \wedge \phi_{g_2}$.

The analysis of $\phi_{\texttt{pen1}}$ is shown in Fig. 4 and 4. Fig. 4 (a) and (b) compare the mean reward and absolute value of maximum angles generated during the learning process. Fig. 4 (a) shows that safe modular learning with exploration guidance is more efficient for finding the optimal policies.

Autonomous Vehicle. We tested our algorithms on controlling a car-like model. Let $s = [p_x, p_y, \theta, v]^T$, $u = [a, \omega]^T$, and L denote the state (position, heading, velocity), control variables (acceleration, steering angle), and length of the vehicle, respectively. The dynamics of the model is $\dot{s} = [v\cos\theta, v\sin\theta, \frac{\tan\omega}{L}, K_u a]^T$ where K_u is the physical constant. To model uncertainty, we set 25% error in the parameters L and K_u and add Gaussian noise to the accelerations.

Particle Gym We first test our algorithm for the Particle Gym

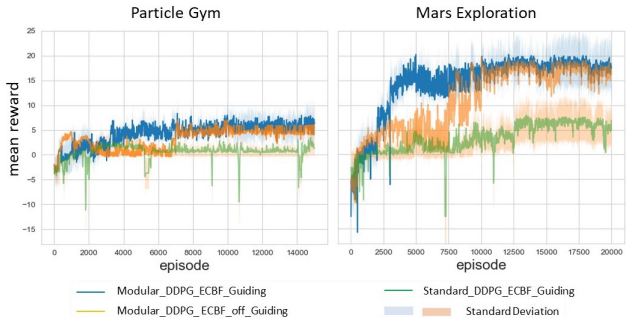

Figure 6: Mean reward during training for particle gym (Left) and Mars exploration (Right) for LTL tasks φ_{Gym_1} and φ_{V_1}.

as shown in Fig. 5. The two missions require the autonomous vehicle (red circle) to sequentially visit the blue regions numbered 1 to 5 over infinite and finite horizons, respectively. The safety constraint is to always avoid the black obstacles and stay within the rectangular workspace, which is encoded as multiple decentralized ECBFs. The LTL task over the infinite horizon is $\phi_{Gym_1} = \Box((\Diamond \mathtt{R}_1 \wedge \Diamond(\mathtt{R}_2 \wedge \Diamond \ldots \wedge \mathtt{R}_5))) \wedge \Box\phi_{\mathtt{C}_3}$. The results of mean reward collection for the task φ_{Gym_1} during training compared with two baselines are shown in Fig. 6 (left), which illustrates better performance of the modular architecture and effectiveness of exploration guiding.

Mars Rover: Lastly, we tested our algorithms in a large scale environment, and used motion planning to complete complex exploration missions using satellite images as shown in Fig. 5 (Right). The missions are to explore areas around the Victoria Crater. In Fig. 5 (Right), the mission requires visiting all spots along the path of the Mars rover Opportunity, and avoiding the unsafe areas (red circle). The LTL specification of the mission over the infinite horizon is

$$\phi_{V_1} = \Box\left(\Diamond \mathtt{V}_1 \wedge \Diamond\left(\mathtt{V}_2 \wedge \Diamond(\ldots \wedge \Diamond(\mathtt{V}_{10} \wedge \Diamond \mathtt{V}_{\mathtt{Start}}))\right)\right) \wedge \Box\phi_{\mathtt{C}_4},$$

where $\mathtt{V}_\mathtt{i}$ denotes the i-th target (blue spot) numbered from bottom-left to top-right. $\varphi_{\mathtt{C}_4}$ represents safety requirements (barrier functions) s.t. the agent always avoids the unsafe crater area marked with a red circle in Fig. 5. The description of the overall task in English is "visit the targets 1 to 10 and then return to the start position, repetitively, while avoiding the unsafe regions". Similarly, the finite horizon task is $\phi_{V_2} = \Diamond \mathtt{V}_1 \wedge \Diamond(\mathtt{V}_2 \wedge \Diamond(\ldots \wedge \Diamond(\mathtt{V}_{10} \wedge \Diamond \mathtt{V}_{\mathtt{Start}}) \ldots) \wedge \Box\phi_{\mathtt{C}_4}$. The results of mean reward collection for the task φ_{V_1} during training compared with two baselines are shown in Fig. 6 (right). It shows the better performance and effectiveness of safe modular DDPG with exploration guiding enabled.

VI. Conclusions

This paper introduces a model-free deep reinforcement learning (DRL) framework for continuous control, aiming to fulfill LTL specifications through automaton-based rewards and a modular architecture. To ensure safe exploration, we incorporate ECBF and GPs as a monitoring and correction mechanism for RL controllers. We also propose a novel method by integrating LTL automata's sink components to enhance exploration and learning efficiency. Rigorous analysis confirms

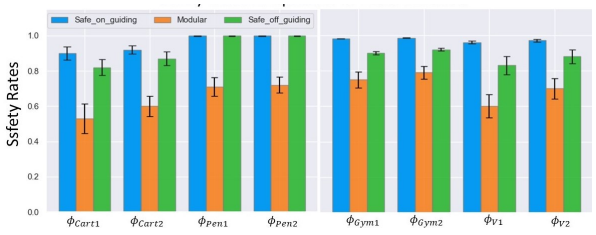

Figure 7: Safety rates analysis for each environments that includes corresponding LTL tasks. The results are all conducted over 10 independent learning trials.

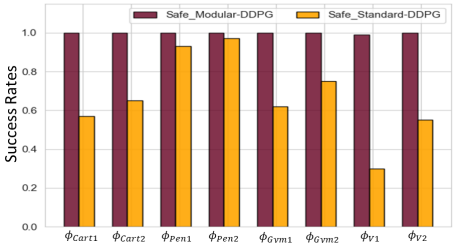

Figure 8: Performance evaluation through success rates.

that the safe filter and exploration guidance maintain desired behaviors and do not disrupt original optimal solutions. The algorithm is implemented in various control systems and compared with baselines to showcase its benefits. Future research will explore multi-agent cooperative tasks and bridge the gap between simulations and real-world applications

REFERENCES

[1] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.

[2] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust region policy optimization," in *International conference on machine learning*. PMLR, 2015, pp. 1889–1897.

[3] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *Int. Conf. Learn. Represent.*, San Juan, Puerto rico, 2016.

[4] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2016.

[5] J. Choi, F. Castaneda, C. J. Tomlin, and K. Sreenath, "Reinforcement learning for safety-critical control under model uncertainty, using control lyapunov functions and control barrier functions," *arXiv preprint arXiv:2004.07584*, 2020.

[6] F. Castañeda, J. J. Choi, B. Zhang, C. J. Tomlin, and K. Sreenath, "Pointwise feasibility of gaussian process-based safety-critical control under model uncertainty," *arXiv preprint arXiv:2106.07108*, 2021.

[7] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig, "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 5, pp. 411–444, 2022.

[8] J. Garcıa and F. Fernández, "A comprehensive survey on safe reinforcement learning," *Journal of Machine Learning Research*, vol. 16, no. 1, pp. 1437–1480, 2015.

[9] M. W. Seeger, S. M. Kakade, and D. P. Foster, "Information consistency of nonparametric gaussian process methods," *IEEE Transactions on Information Theory*, vol. 54, no. 5, pp. 2376–2382, 2008.

[10] J. F. Fisac, A. K. Akametalu, M. N. Zeilinger, S. Kaynama, J. Gillula, and C. J. Tomlin, "A general safety framework for learning-based control in uncertain robotic systems," *IEEE Transactions on Automatic Control*, 2018.

[11] R. Cheng, G. Orosz, R. M. Murray, and J. W. Burdick, "End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 3387–3395.

[12] V. Dhiman, M. J. Khojasteh, M. Franceschetti, and N. Atanasov, "Control barriers in bayesian learning of system dynamics," *IEEE Transactions on Automatic Control*, 2021.

[13] Y. Emam, P. Glotfelter, Z. Kira, and M. Egerstedt, "Safe model-based reinforcement learning using robust control barrier functions," *arXiv preprint arXiv:2110.05415*, 2021.

[14] M. Kloetzer and C. Belta, "Automatic deployment of distributed teams of robots from temporal logic motion specifications," *IEEE Transactions on Robotics*, vol. 26, no. 1, pp. 48–61, 2009.

[15] X. Luo, Y. Kantaros, and M. M. Zavlanos, "An abstraction-free method for multirobot temporal logic optimal control synthesis," *IEEE Transactions on Robotics*, 2021.

[16] R. T. Icarte, T. Klassen, R. Valenzano, and S. McIlraith, "Using reward machines for high-level task specification and decomposition in reinforcement learning," in *International Conference on Machine Learning*, 2018, pp. 2107–2116.

[17] A. Camacho, R. T. Icarte, T. Q. Klassen, R. A. Valenzano, and S. A. McIlraith, "LTL and beyond: Formal languages for reward function specification in reinforcement learning." in *IJCAI*, vol. 19, 2019, pp. 6065–6073.

[18] M. Hasanbeig, Y. Kantaros, A. Abate, D. Kroening, G. J. Pappas, and I. Lee, "Reinforcement learning for temporal logic control synthesis with probabilistic satisfaction guarantees," in *2019 IEEE 58th Conference on Decision and Control (CDC)*. IEEE, 2019, pp. 5338–5343.

[19] A. K. Bozkurt, Y. Wang, M. M. Zavlanos, and M. Pajic, "Control synthesis from linear temporal logic specifications using model-free reinforcement learning," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 10 349–10 355.

[20] M. Cai, M. Hasanbeig, S. Xiao, A. Abate, and Z. Kan, "Modular deep reinforcement learning for continuous motion planning with temporal logic," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 7973–7980, 2021.

[21] X. Li, Z. Serlin, G. Yang, and C. Belta, "A formal methods approach to interpretable reinforcement learning for robotic planning," *Science Robotics*, vol. 4, no. 37, 2019.

[22] M. Srinivasan and S. Coogan, "Control of mobile robots using barrier functions under temporal logic specifications," *IEEE Transactions on Robotics*, vol. 37, no. 2, pp. 363–374, 2020.

[23] P. Jagtap, S. Soudjani, and M. Zamani, "Formal synthesis of stochastic systems via control barrier certificates," *IEEE Transactions on Automatic Control*, vol. 66, no. 7, pp. 3097–3110, 2020.

[24] C. Baier and J.-P. Katoen, *Principles of model checking*. MIT press, 2008.

[25] M. Kloetzer and C. Belta, "A fully automated framework for control of linear systems from temporal logic specifications," *IEEE Transactions on Automatic Control*, vol. 53, no. 1, pp. 287–297, 2008.

[26] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International Conference on Machine Learning*. PMLR, 2018, pp. 1861–1870.

[27] C. J. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, no. 3-4, pp. 279–292, 1992.

[28] M. Cai, S. Xiao, B. Li, Z. Li, and Z. Kan, "Reinforcement learning based temporal logic control with maximum probabilistic satisfaction," in *Int. Conf. Robot. Autom.* IEEE, 2021, pp. 806–812.

[29] S. Sickert, J. Esparza, S. Jaax, and J. Křetínskỳ, "Limit-deterministic Büchi automata for linear temporal logic," in *Int. Conf. Comput. Aided Verif.* Springer, 2016, pp. 312–332.

[30] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *ICML*, vol. 99, 1999, pp. 278–287.

[31] N. Srinivas, A. Krause, S. M. Kakade, and M. W. Seeger, "Information-theoretic regret bounds for gaussian process optimization in the bandit setting," *IEEE transactions on information theory*, vol. 58, no. 5, pp. 3250–3265, 2012.

[32] Q. Nguyen and K. Sreenath, "Exponential control barrier functions for enforcing high relative-degree safety-critical constraints," in *American Control Conference (ACC)*. IEEE, 2016, pp. 322–328.

[33] A. Robey, H. Hu, L. Lindemann, H. Zhang, D. V. Dimarogonas, S. Tu, and N. Matni, "Learning control barrier functions from expert demonstrations," in *2020 59th IEEE Conference on Decision and Control (CDC)*. IEEE, 2020, pp. 3717–3724.

[34] M. Cai and C.-I. Vasile, "Safety-critical learning of robot control with temporal logic specifications," *arXiv preprint arXiv:2109.02791*, 2021.